



TAIZ UNIVERSITY

Faculty of Engineering and Information Technology

Software Engineering

PROJECT DOCUMENTATION

Enterprise-Grade Learning Management System (LMS Platform)

Prepared by:

Mohammed Mahyoub Abdu Mohammed
Mortada Abdullah Mohammed Abdullah
Nader Mogib Mohammed
Hamdi Mahmoud Mohammed Noman
EzzAldeen Nabil Sarhan
Mohammed Faisal Mohammed Mahyoub

Under the Supervision of:

Eng. Anas Alsabiry

Academic Year: 2026

1. Introduction

1.1 Project Overview

The Enterprise Learning Management System (LMS) is a full-stack web application developed as a graduation project for the Faculty of Computer Science and Information Technology at Taiz University. The system is designed to provide a comprehensive, professional-grade platform for delivering, managing, and tracking e-learning content in Arabic-speaking educational environments.

The platform supports three distinct user roles — Super Admin, Instructor, and Student — and is built on modern web technologies including Laravel 12, React 19 with TypeScript, Livewire 3, Inertia.js, and Tailwind CSS. It includes advanced features such as AI-powered tutoring via the Gemini API, interactive video quizzes, gamification mechanics, certificate generation with QR code verification, and a real-time notification system.

1.2 Problem Statement

Higher-education institutions in Arab countries frequently depend on paper-based or outdated digital systems that are unable to meet the demands of modern e-learning. Key challenges include:

- Absence of a unified platform that supports Arabic (RTL) content natively.
- Lack of tools for instructors to create, manage, and monetise structured course content.
- No mechanism for automated assessment, grading, and academic certificate issuance.
- Limited student engagement without gamification or AI-assisted learning.
- No administrative oversight workflow for instructor verification and course approval.

1.3 Objectives

The primary objectives of this project are:

- Develop a multi-role e-learning platform that is fully compliant with Arabic RTL requirements.
- Implement a structured course hierarchy (Course → Section → Lesson) with media management.
- Build a secure quiz engine supporting multiple question types, timed assessment, and auto-grading.
- Integrate Google Gemini AI as a context-aware tutoring chatbot for students.
- Provide certificate generation (PDF + QR verification) upon successful course completion.
- Implement gamification (XP points and levels) to motivate student engagement.
- Deliver a financial management module for instructor earnings and course pricing.

1.4 Scope

The system covers all core LMS functionalities as described below. Explicitly out of scope for this version are: a mobile native application, video conferencing/live-streaming, third-party payment gateway integration (replaced by an internal balance system), and a Filament-based admin panel (the SRS mentioned Filament, but the implementation uses custom Blade/React admin pages with Livewire).

2. System Overview

2.1 User Roles

Role	Description	Key Capabilities
Super Admin	System administrator	Approve/reject instructor applications, manage categories, view all data
Instructor	Content creator	Create courses, manage sections/lessons, add quizzes, view financials
Student	Learner	Browse & purchase courses, watch videos, take quizzes, earn certificates & XP

2.2 Main Services

- Course Management — full CRUD for courses, sections, and video lessons.
- Student Learning — sequential lesson unlocking, video progress tracking (second-level precision).
- Assessment Engine — timed quizzes, auto-grading, practice mode, final exams.
- Certificate System — PDF certificate generation (mPDF) with UUID and QR code verification.
- AI Tutoring — context-aware chatbot powered by Google Gemini 2.5 Flash.
- Gamification — XP transactions, level computation, real-time XP notifications.
- Financial Module — course pricing, student balance, instructor earnings dashboard.
- Instructor KYC — multi-step application with document upload and admin review.
- Real-time Notifications — Laravel Reverb (WebSocket) for in-app alerts.

2.3 Technology Stack

Layer	Technology	Version
Backend Framework	Laravel	12.x
PHP Runtime	PHP	8.2+
Database	MySQL	8.0+
Frontend Framework	React + TypeScript	19.x / 6.x
SPA Bridge	Inertia.js	3.x
CSS Framework	Tailwind CSS	3.x
Reactive Components	Livewire	3.6.x
Lightweight Interactivity	Alpine.js	3.15.x
Build Tool	Vite	6.x
Role/Permission	Spatie Laravel Permission	6.x
Media Management	Spatie Media Library	11.x
PDF Generation	mPDF	8.3
QR Code	Simple QrCode	4.2
AI Integration	Google Gemini API (2.5 Flash)	v1beta
WebSocket	Laravel Reverb	1.x

3. System Architecture

3.1 Architectural Pattern

The system follows a layered MVC architecture enhanced with the Repository Pattern and Service Layer Pattern to ensure separation of concerns and testability. The frontend uses a Single-Page Application (SPA) model via Inertia.js, which bridges Laravel Blade with React components without requiring a separate REST API. Livewire 3 is used alongside Inertia for components that require server-side reactivity without full page navigation (e.g., course catalog search, the AI chatbot, and the quiz player).

3.2 Layer Description

Layer	Components	Responsibility
Presentation	React/TSX Pages, Blade Views, Livewire Views	UI rendering, user interaction
Controller	Http/Controllers/* (Admin, Instructor, Student)	Request handling, input validation, response
Service	Services/* (QuizService, CertificateService, GamificationService, GeminiAiService)	Complex business logic, orchestration
Repository	Repositories/Eloquent/* (BaseRepository + domain repos)	Data access abstraction, query logic
Model	Models/* (Course, Lesson, Quiz, Certificate, etc.)	Eloquent ORM, relationships, casts
Database	MySQL via Eloquent ORM	Data persistence, migrations

3.3 Architecture Diagram

Figure 3.1 — System Architecture (Mermaid notation)

```
graph TD
    Browser["Browser / Client"]
    subgraph "Laravel Application"
        Inertia["Inertia.js Adapter"]
        LW["Livewire 3 Components"]
        Routes["routes/web.php (role-based middleware)"]
        Controllers["Http/Controllers (Admin | Instructor | Student)"]
        Services["Service Layer (Quiz | Certificate | AI | Gamification)"]
        Repositories["Repository Layer (Eloquent)"]
        Models["Eloquent Models"]
        Middleware["SetSecurityHeaders | role | auth | verified"]
    end
```

```
end
React["React 19 + TypeScript (Vite)"]
MySQL["MySQL 8.0"]
Storage["Spatie Media Library (File Storage)"]
Gemini["Google Gemini AI API"]
Reverb["Laravel Reverb (WebSocket)"]

Browser --> Inertia
Browser --> LW
Inertia --> React
Inertia --> Routes
LW --> Routes
Routes --> Middleware
Middleware --> Controllers
Controllers --> Services
Services --> Repositories
Repositories --> Models
Models --> MySQL
Services --> Gemini
Services --> Storage
Reverb --> Browser
```

4. Database Design

4.1 Overview

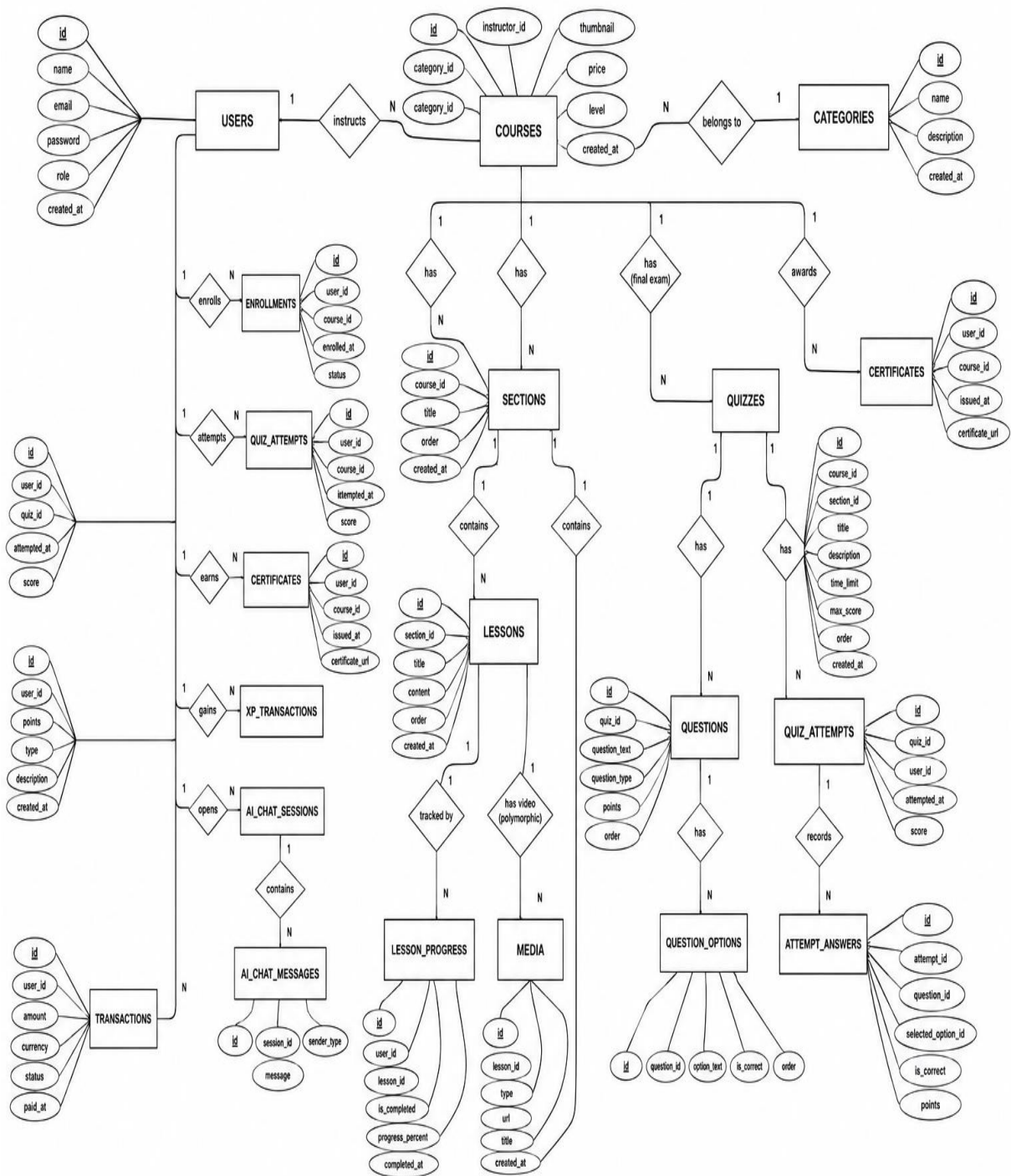
The MySQL database contains 20 tables, all created via Laravel migrations with strict foreign-key constraints. The schema supports the full course lifecycle, user role management (via Spatie Permission tables), media attachments (via Spatie Media Library), quizzes, certificates, XP gamification, AI chat sessions, real-time notifications, and financial transactions.

4.2 Core Tables

Table	Purpose	Key Columns
users	All platform users	id, name, email, password, balance, avatar, instructor_status, bio, expertise, phone, id_front_path, id_back_path, selfie_path
courses	Course catalogue	id, instructor_id, category_id, title, slug, description, price, difficulty (Enum), status (Enum)
sections	Course modules	id, course_id, title, description, order
lessons	Individual lessons	id, section_id, title, content, order
quizzes	Section quizzes & final exams	id, section_id, course_id, title, time_limit_minutes, is_practice, is_final_exam, order
questions	Quiz questions	id, quiz_id, type (Enum), stem, points, video_timestamp
question_options	MCQ options	id, question_id, text, is_correct
quiz_attempts	Student attempts	id, user_id, quiz_id, started_at, completed_at, score, is_passed
attempt_answers	Per-question answers	id, quiz_attempt_id, question_id, question_option_id, answer_text, is_correct, points_awarded
enrollments	Student enrolments	id, user_id, course_id, enrolled_at, completed_at
lesson_progress	Progress tracking	id, user_id, lesson_id, is_completed, last_position_seconds
certificates	Issued certificates	id, uuid, user_id, course_id, issued_at
xp_transactions	Gamification XP	id, user_id, amount, description
transactions	Financial records	id, user_id, course_id, amount, type
ai_chat_sessions	AI chat sessions	id, user_id, course_id, lesson_id, title
ai_chat_messages	AI messages	id, ai_chat_session_id, role, content
notifications	Laravel notifications	id, type, notifiable_id, data, read_at
media	Spatie media entries	id, model_type, model_id, collection_name, file_name, disk
categories	Course categories	id, name, slug, icon
permissions / roles / model_has_roles	Spatie role tables	Standard Spatie Permission schema

4.3 Entity Relationship Diagram

Figure 4.1 — Core Entity Relationships (Mermaid erDiagram)



5. System Modules

5.1 Authentication Module

Authentication is implemented using Laravel Breeze scaffolding adapted for a dual-mode frontend: Livewire Volt handles Blade-based auth pages (login, register, password reset, email verification), while Inertia.js serves the React SPA pages after login. Key characteristics:

- Session-based authentication with encrypted cookies (SESSION_ENCRYPT=true).
- Email verification required before accessing protected routes (middleware: verified).
- Password reset via email link with secure token validation.
- Bcrypt hashing with 12 rounds configured in .env (BCRYPT_ROUNDS=12).

5.2 Role & Permission Management

Spatie Laravel Permission v6 manages access control. Three roles are seeded at boot: "Super Admin", "Instructor", and "Student". All protected route groups use the role middleware (e.g., middleware: ["auth", "role:Super Admin"]). The User model uses the HasRoles trait, enabling per-user role checks throughout controllers and Livewire components.

5.3 Instructor KYC & Application Module

A multi-step instructor on-boarding flow (ApplicationController) allows any authenticated user to apply as an instructor. The flow captures bio, expertise, phone number, and uploads of identification documents (front, back, selfie) via Spatie Media Library. The InstructorStatus enum tracks states: pending, approved, rejected. Admins review applications via a dedicated React page (InstructorApplications.tsx) and approve or reject with an optional rejection reason.

5.4 Course Management Module

Instructors manage courses through a two-page flow: CourseCreate for initial metadata (title, description, difficulty, price, category, thumbnail) and CourseBuilder for the structural content (sections, lessons, quizzes). The CourseStatus enum (Draft → PendingApproval → Published) controls course visibility. The Instructor/CourseController handles all CRUD operations, delegating to CourseService which interacts with the CourseRepository.

5.5 Video Lesson & Progress Module

Each lesson may have a video attachment managed by Spatie Media Library under the "lesson_video" collection. The CoursePlayer Livewire component generates time-limited signed URLs

(valid 2 hours) for video streaming, preventing direct file access. Students must complete lessons sequentially — the `calculateUnlockedLessons()` method enforces this by computing which lessons are unlocked based on prior completion status. The last playback position is saved to the `lesson_progress` table at the second level via an AJAX endpoint, and restored on the next visit.

5.6 Quiz & Assessment Module

The `QuizService` orchestrates the full quiz lifecycle. Instructors configure quizzes at section level or as a final exam at course level (`is_final_exam` flag). Four question types are supported via the `QuestionType` enum: `single_choice`, `multiple_choice`, `true_false`, and `short_text` (the last requires manual grading and is currently not auto-scored). Quizzes may have an optional countdown timer (`time_limit_minutes`). Practice mode (`is_practice`) does not affect the student XP score. On submission, `QuizService` auto-grades objective questions, awards XP, and — if the quiz is a final exam and the student passes — automatically issues a certificate.

5.7 Certificate Module

`CertificateService` generates a PDF certificate using `mPDF` (UTF-8, A4-Landscape, zero margins) from a Blade template (`certificates/template.blade.php`) with a custom background image. Each certificate record stores a unique UUID. A Simple `QrCode` image is embedded in the PDF pointing to the public verification URL (`/verify/{uuid}`), where anyone can confirm the authenticity, course name, issue date, and student name without logging in.

5.8 Gamification Module

`GamificationService` awards XP to students via the `xp_transactions` table upon passing quizzes (`XP = score`). Total XP and computed level ($XP / 500 + 1$) are exposed as dynamic model attributes on `User`. An `XpAwardedNotification` is dispatched to the student notification channel when XP is earned. The student dashboard displays current XP, level, and a progress bar toward the next level.

5.9 AI Chatbot Module

`GeminiAiService` integrates with the Google Gemini 2.5 Flash model via direct HTTP calls using Laravel's `Http` facade. The chatbot is rendered as a floating widget (`AiChatbot Livewire` component) in the course learning interface. Each conversation is persisted as an `AiChatSession` with child `AiChatMessages`. The system prompt is context-aware, injecting the current course and lesson titles to focus the AI exclusively on the relevant learning material. A `MockAiService` is provided for development environments without an API key.

5.10 Admin Panel

The Super Admin panel uses custom React/Inertia pages (not Filament, contrary to the SRS). Functionality includes: category management with a Livewire CategoryManager component providing live create/delete, and an instructor applications management page with tabbed filtering (pending, approved, rejected) and approve/reject actions with optional rejection notes.

5.11 Financial Module

Courses may be priced (decimal price field on courses). Students purchase courses through a Checkout flow (CheckoutController / Checkout Livewire) that deducts from the student's account balance and transfers the amount to the instructor's balance via a Transaction record. The Instructor Financials page (FinancialController / Financials Livewire) displays total earnings, monthly breakdowns, and individual transaction history.

6. Implementation

6.1 Project Directory Structure

```

app/
  Actions/           # Standalone action classes
  DTOs/             # Data Transfer Object base
  Enums/            # PHP 8.2 backed enums (CourseStatus, CourseDifficulty,
  QuestionType, InstructorStatus)
  Http/
    Controllers/
      Admin/         # CategoryController, InstructorApplicationsController
      Auth/          # Password reset, email verification
      Instructor/    # ApplicationController, CourseController, QuizController,
  FinancialController
      Student/       # AiChatController, CheckoutController, CourseCatalogController,
                        # CourseLearnController, QuizTakingController
    Middleware/      # SetSecurityHeaders
  Livewire/
    Admin/           # CategoryManager
    Instructor/      # CourseBuilder, CourseManager, QuizManager, Financials
    Student/         # AiChatbot, Checkout, CourseCatalog, CoursePlayer, Dashboard,
                        # NotificationDropdown, QuizTaking
  Models/           # 16 Eloquent models
  Repositories/
    Contracts/       # Repository interfaces
    Eloquent/        # BaseRepository + 6 domain repositories
  Services/
    AI/              # AiServiceInterface, GeminiAiService, MockAiService
    Assessment/      # CertificateService, QuizService
    Courses/         # CategoryService, CourseService, LessonService, SectionService
    Gamification/    # GamificationService
  resources/js/      # React/TypeScript frontend
    components/      # Reusable UI: AiChatbot.tsx, CourseCard.tsx, ui/Button, Card,
  Input
    layouts/         # AppLayout.tsx, GuestLayout.tsx
    pages/
      admin/         # CategoryManager.tsx, InstructorApplications.tsx
      auth/          # Login, Register, ForgotPassword, ResetPassword
      instructor/    # Apply, CourseBuilder, CourseCreate, CourseManager,
                        # Financials, InstructorDashboard, QuizManager
      student/       # Checkout, CourseCatalog, CoursePlayer, CourseShow, QuizTaking
  types/index.ts     # TypeScript interfaces shared across pages

```

6.2 Key Controllers

Controller	Methods	Notes
Instructor/CourseController	index, create, store, update, storeSection, storeLesson	Full course/section/lesson CRUD
Instructor/QuizController	storeSectionQuiz, storeFinalExam, manage, storeQuestion, updateTimer, destroyQuestion	Quiz and question management
Student/CourseLearnController	show, markCompleted	Video player with signed URL, progress update
Student/QuizTakingController	show, start, submit	Delegates to QuizService
Student/AiChatController	index, store	AI session management + Gemini call
CertificateController	download, verify	PDF download + public QR verification
Admin/InstructorApplicationsController	index, approve, reject	KYC approval workflow

6.3 Key Livewire Components

Component	Function
Student/CoursePlayer	Signed video URL generation, sequential lesson unlock, interactive video question rendering, progress save
Student/QuizTaking	Real-time countdown timer, question navigation, answer collection, auto-submit on expiry
Student/AiChatbot	Floating chat widget, session management, Gemini response display
Student/CourseCatalog	Live search and filter of courses without page reload
Instructor/CourseBuilder	Drag-and-drop-style course structure editor (sections, lessons, quizzes)
Instructor/QuizManager	Question CRUD, question type selection, option management
Admin/CategoryManager	Live category create/delete with instant feedback

6.4 PHP 8.2 Features Used

- Backed Enums — CourseStatus, CourseDifficulty, QuestionType, InstructorStatus all use PHP 8.2 string-backed enums.
- declare(strict_types=1) — enforced in all Service and Middleware files.
- readonly Properties — used in BaseRepository (\$model is readonly).
- Named Arguments and Constructor Property Promotion — used in Service constructors.

7. Security

7.1 Authentication & Session Security

- Session encryption enabled (SESSION_ENCRYPT=true) with secure-only cookies in production.
- HttpOnly and SameSite=Lax attributes on session cookies.
- Bcrypt password hashing with 12 rounds.
- Email verification required (MustVerifyEmail interface on User model).

7.2 Authorization

- Role-based middleware on all route groups: role:Super Admin, role:Instructor, role:Student|Super Admin|Instructor.
- Controller-level ownership checks (instructor can only edit their own courses).
- The VerifiesEnrollment trait in CoursePlayer confirms student enrollment before serving video content.

7.3 HTTP Security Headers (SetSecurityHeaders Middleware)

Header	Value / Purpose
Content-Security-Policy	Restricts script/style/image origins; nonce-based for inline scripts; frame-ancestors 'none'
X-Frame-Options	DENY — prevents clickjacking
X-Content-Type-Options	nosniff — prevents MIME-type sniffing
Referrer-Policy	strict-origin-when-cross-origin
Permissions-Policy	Disables camera, microphone, geolocation, payment, USB
Strict-Transport-Security	max-age=31536000; includeSubDomains (production only)
X-XSS-Protection	1; mode=block (legacy browsers)
Server / X-Powered-By	Removed — server fingerprint hidden

7.4 Media & Content Protection

- Video files are served via time-limited Signed URLs (2-hour expiry) generated by Laravel's URL::signedRoute. Direct file paths are never exposed to the browser.
- Spatie Media Library stores files outside the public web root (storage/app/private).
- Quiz copying prevention is handled at the frontend level (quiz-taking page disables text selection).

7.5 Input Validation

- All form inputs are validated using Laravel Form Requests or inline validation rules within controllers.
- CSRF protection is automatically applied to all POST/PATCH/DELETE routes via Laravel middleware.
- Livewire automatically validates component properties before actions.

8. Testing

8.1 Testing Approach

The project uses PHPUnit 11 integrated with Laravel's test framework. Tests are located under tests/Feature/ and tests/Unit/. The feature tests cover authentication flows using the RefreshDatabase trait and Livewire Volt testing utilities.

8.2 Implemented Test Coverage

Test File	Scenarios Covered
Auth/AuthenticationTest.php	Login screen renders, valid login, invalid password rejection, logout, navigation render
Auth/EmailVerificationTest.php	Verification page renders, email can be resent, redirect after verification
Auth/PasswordResetTest.php	Reset link screen renders, reset link send, password reset with valid token
Auth/PasswordUpdateTest.php	Password update with correct current password, rejection on wrong password
Auth/PasswordConfirmationTest.php	Confirmation screen renders, correct/wrong password confirmation
Auth/RegistrationTest.php	Registration screen renders, new user can register
Feature/ProfileTest.php	Profile page renders, information update, account deletion
Feature/ExampleTest.php	Basic application health check

8.3 Detected Limitations & Gaps

- No dedicated feature tests exist for course management, quiz engine, certificate generation, or the AI chatbot — these modules are untested at the automated level.
- Short-text (open-ended) quiz questions are stored but auto-grading is not implemented; they are saved with `is_correct = null` and require manual review (not yet built in the admin panel).
- The Leaderboard page mentioned in the SRS is not yet implemented; XP data is collected but a public ranking page does not exist.
- Interactive Video Questions (pausing video to ask a question at a timestamp) are partially supported — the data model (video_timestamp column on questions) and CoursePlayer interactiveQuestions array exist, but the complete frontend implementation in the video player UI requires further refinement.
- Dark Mode and full RTL/Arabic UI are specified in the SRS but the current frontend (React/TypeScript via Inertia) uses primarily LTR Tailwind classes; RTL support is planned but not fully activated.

8.4 SRS vs. Implementation Comparison

SRS Requirement	Implementation Status
Laravel 12 / PHP 8.2 backend	Implemented (Laravel 12, PHP ^8.2)
TALL Stack (Blade + Tailwind + Alpine + Livewire)	Partial — Livewire + Alpine used alongside React/Inertia (hybrid approach)
Full Arabic RTL support	Partial — AR locale set, Livewire views have RTL; React pages mostly LTR
Spatie Permission (roles)	Fully implemented
Spatie Media Library (uploads)	Fully implemented
Course lifecycle (Draft → Pending → Published)	Implemented via CourseStatus Enum
Course → Section → Lesson hierarchy	Fully implemented
Timed quiz with countdown timer	Implemented (QuizTaking Livewire)
Multiple question types	3 auto-graded + 1 manual (short_text)
Certificate with UUID + QR Code	Implemented (mPDF + Simple QrCode)
Gamification (XP + level)	Implemented (GamificationService)
Leaderboard page	Not yet implemented
AI Chatbot (Gemini)	Implemented (GeminiAiService)
Interactive Video Questions	Partial — data model ready, UI needs refinement
Signed video URLs	Implemented (2-hour signed URLs)
Financial module (pricing, balance, earnings)	Implemented
Dark Mode	Not yet implemented
Filament admin panel	Not used — replaced with custom React/Livewire admin pages
Redis caching	Config present but CACHE_STORE=database in default .env
Real-time notifications	Laravel Reverb + XpAwardedNotification

9. Deployment

9.1 System Requirements

Component	Minimum Requirement
PHP	8.2 or higher with extensions: PDO, mbstring, openssl, tokenizer, xml, ctype, json, BCMath, fileinfo, GD or Imagick
Composer	2.x
Node.js	18.x or higher
npm	9.x or higher
MySQL	8.0 or higher
Web Server	Apache (with mod_rewrite) or Nginx
Storage	Read/write access to storage/ and bootstrap/cache/
Optional	Redis (for production caching), Laravel Reverb (for WebSockets)

9.2 Installation Steps

Step 1 — Clone the repository and install PHP dependencies:

```
git clone <repository-url> && cd Learning-Management-System-main
composer install --optimize-autoloader --no-dev
```

Step 2 — Install JavaScript dependencies and build frontend assets:

```
npm install
npm run build
```

Step 3 — Configure environment:

```
cp .env.example .env
php artisan key:generate
```

Step 4 — Configure database credentials in .env:

```
DB_DATABASE=lms
DB_USERNAME=lms_user
DB_PASSWORD=YOUR_STRONG_PASSWORD
```

Step 5 — Run database migrations and seed default data:

```
php artisan migrate:fresh --seed
```

Step 6 — Link public storage:

```
php artisan storage:link
```

9.3 Environment Configuration

The following .env variables are critical for full functionality:

Variable	Purpose	Example Value
APP_URL	Public application URL	https://lms.example.com
APP_LOCALE	Default locale	ar
APP_TIMEZONE	Timezone	Asia/Aden
DB_*	MySQL connection	See Step 4 above
GEMINI_API_KEY	Google Gemini AI key	YOUR_KEY from Google AI Studio
SESSION_ENCRYPT	Encrypt sessions	true
SESSION_SECURE_COOKIE	HTTPS-only cookies	true (production)
BROADCAST_CONNECTION	WebSocket driver	reverb (production)
CACHE_STORE	Cache backend	database or redis
QUEUE_CONNECTION	Queue backend	database or redis

9.4 Running the Project (Development)

For development, use two concurrent terminal windows:

```
# Terminal 1 - Laravel development server
php artisan serve

# Terminal 2 - Vite asset watcher
npm run dev
```

Access the application at: <http://localhost:8000>

9.5 Default Seeded Accounts

Role	Email	Password
Super Admin	admin@lms.com	password
Instructor	instructor@lms.com	password
Student	student@lms.com	password

10. Conclusion

10.1 Project Achievements

The Enterprise LMS graduation project successfully delivers a production-ready, multi-role e-learning platform built on Laravel 12 and React 19. The following major features were implemented and are fully functional:

- A complete instructor lifecycle from KYC application through course creation to student enrolment.
- A robust quiz engine supporting four question types, configurable time limits, practice mode, and automatic final-exam certificate issuance.
- A certificate generation and public verification system with mPDF and QR codes.
- A Google Gemini 2.5 Flash–powered AI tutoring chatbot with context awareness of the active course and lesson.
- A gamification system that awards XP on quiz completion and computes student levels.
- A financial module enabling course pricing, student balance management, and instructor earnings tracking.
- A hardened security posture with HTTP security headers, signed video URLs, role-based access control, and encrypted sessions.
- A repository pattern and service layer architecture ensuring maintainability and testability.

10.2 Future Improvements

The following enhancements are recommended for future development iterations:

- Complete RTL/Arabic UI — apply `dir="rtl"` and Arabic font families (Cairo, Tajawal) across all React/Inertia pages, not only Livewire views.
- Leaderboard Page — build a public XP ranking page showing top students per course or platform-wide.
- Dark Mode — implement a Tailwind dark-mode toggle using Alpine.js with localStorage persistence.
- Interactive Video Questions — complete the video player overlay UI that pauses video at a timestamp and shows a quiz modal before resuming.
- Short-Text Question Grading — add an admin/instructor review interface for manually grading open-ended quiz answers.
- Automated Test Expansion — add feature tests for course management, quiz engine, certificate generation, and AI chat routes.
- Redis Integration — switch `CACHE_STORE` and `QUEUE_CONNECTION` to Redis for production scalability.

- **Mobile Application** — develop a React Native or Flutter app consuming a dedicated REST API layer.
- **Live Sessions** — integrate video conferencing (e.g., Jitsi or Zoom SDK) for live instructor-led classes.

10.3 Final Summary

This project demonstrates the practical application of modern software engineering principles — including the Repository Pattern, Service Layer, PHP 8.2 backed Enums, Inertia.js SPA bridging, and AI API integration — in the context of a real-world educational platform. Despite some deviations from the original SRS (notably the frontend stack choice and the pending RTL implementation), the core learning management, assessment, certification, and AI tutoring features are fully operational and form a solid foundation for a production deployment.

The system architecture is designed for scalability and maintainability, and the codebase follows clean-code principles (SOLID, PSR-4 autoloading, strict types) that will facilitate future enhancements by the development team.